



Introducción e instalación de Spark

Rafael Pastor Vargas
SCC-ETSI Informática-UNED

INTRODUCCIÓN E INSTALACIÓN DE APACHE SPARK.

- Arquitectura y propósito de Spark.
- Componentes de la pila unificada de Spark.
- Tipos de gestores de clusters de Spark.
- Resilient Distributed Dataset (RDD).
- Instalación de Spark.
- Ejecución/uso de las shell de Spark (Scala/Python).
- El primer programa Spark.
- Ejecución de los ejemplos de Spark.

APACHE SPARK

- https://spark.apache.org/



MAPREDUCE



SPARK

```
public static class WordCountMapClass extends MapReduceBase
  implements Mapper<LongWritable, Text, Text, IntWritable> {

  private final static IntWritable one = new IntWritable(1);
  private Text word = new Text();

  public void map(LongWritable key, Text value,
                  OutputCollector<Text, IntWritable> output,
                  Reporter reporter) throws IOException {
    String line = value.toString();
    StringTokenizer itr = new StringTokenizer(line);
    while (itr.hasMoreTokens()) {
      word.set(itr.nextToken());
      output.collect(word, one);
    }
  }
}

public static class WorkdCountReduce extends MapReduceBase
  implements Reducer<Text, IntWritable, Text, IntWritable> {

  public void reduce(Text key, Iterator<IntWritable> values,
                     OutputCollector<Text, IntWritable> output,
                     Reporter reporter) throws IOException {
    int sum = 0;
    while (values.hasNext()) {
      sum += values.next().get();
    }
    output.collect(key, new IntWritable(sum));
  }
}
```

```
val file = spark.textFile("hdfs://...")
val counts = file.flatMap(line => line.split(" "))
                 .map(word => (word, 1))
                 .reduceByKey(_ + _)
counts.saveAsTextFile("hdfs://...")
```

APACHE SPARK (1/4)



- Framework distribuido y altamente escalable para la computación paralela (**en memoria**) de gran cantidad de datos
  - Paralelismo de datos (datasets particionados computados en paralelo)
  - Soporte para varios lenguajes: Scala, Java, Phyton y R
  - Más rápido que MapReduce
  - Algoritmos/Queries interactivas, streaming (no solo procesos batch/Jobs)
- Componentes fundamentales (Spark Core)
  - RDD (Resilent Distributed Datasets)
    - Transformaciones y acciones sobre los RDDs
  - Cluster Manager

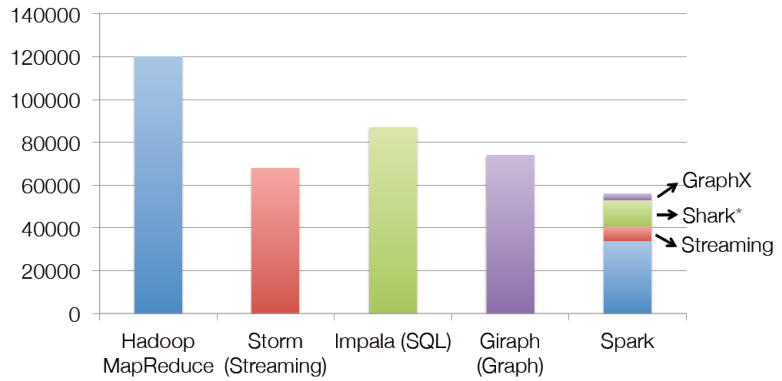
APACHE SPARK (2/4)

Brief History of Spark

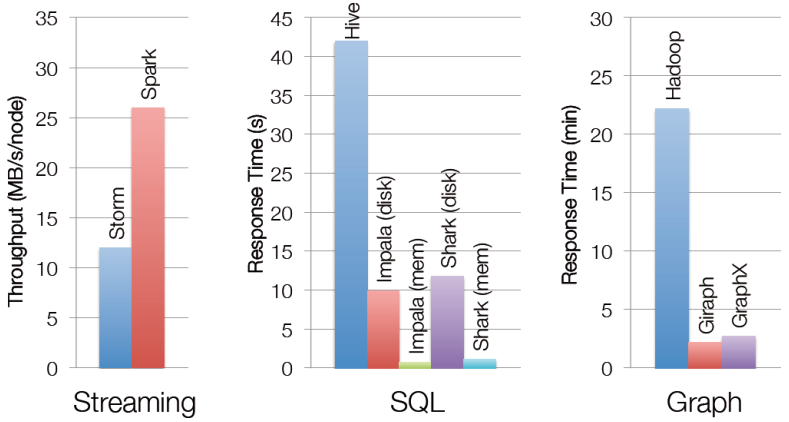
- 2002 – MapReduce @ Google
- 2004 – MapReduce paper
- 2006 – Hadoop @ Yahoo
- 2008 – Hadoop Summit
- 2010 – Spark paper
- 2014 – Apache Spark top-level

- The State of Spark, and Where We're Going Next
  - https://databricks.com/session/the-state-of-spark-and-where-were-going-next

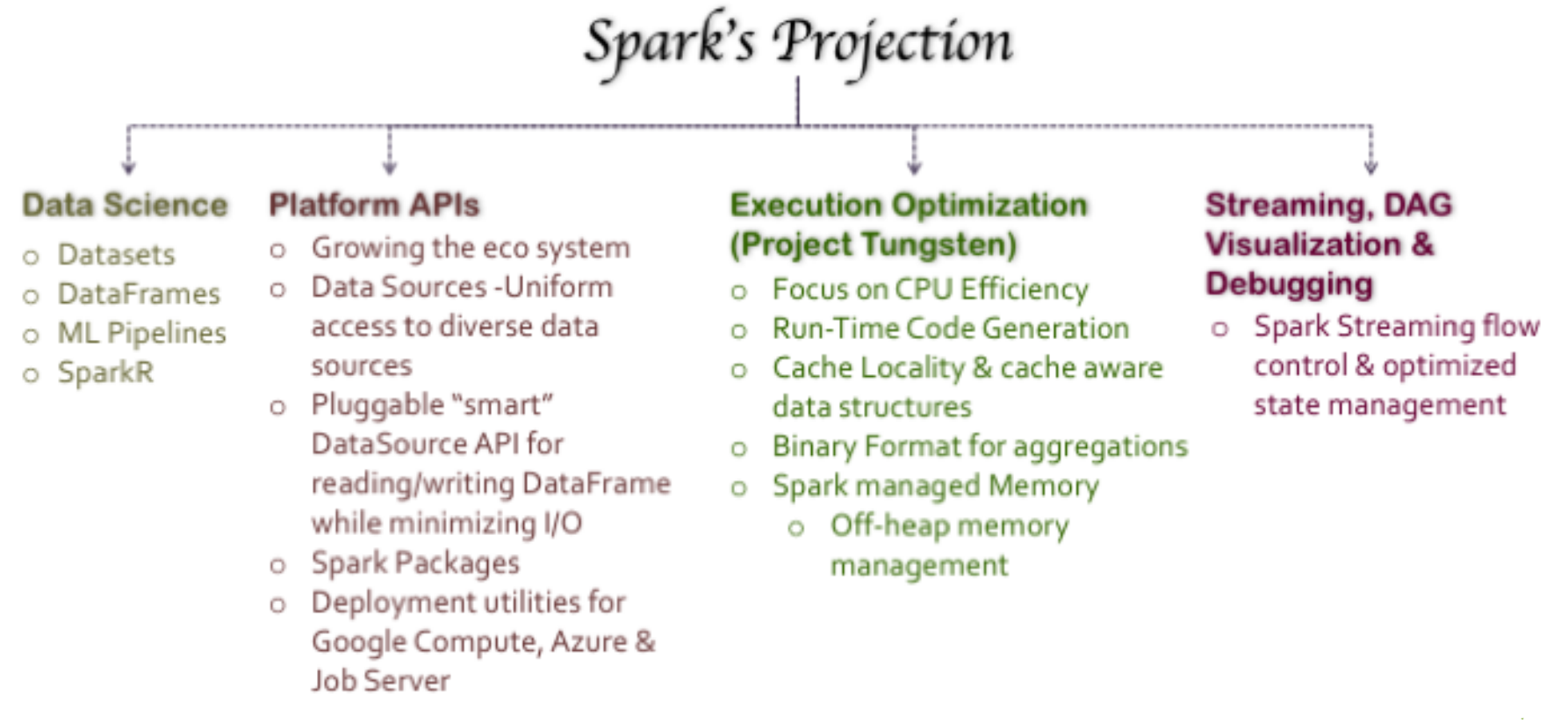
Code Size



Performance

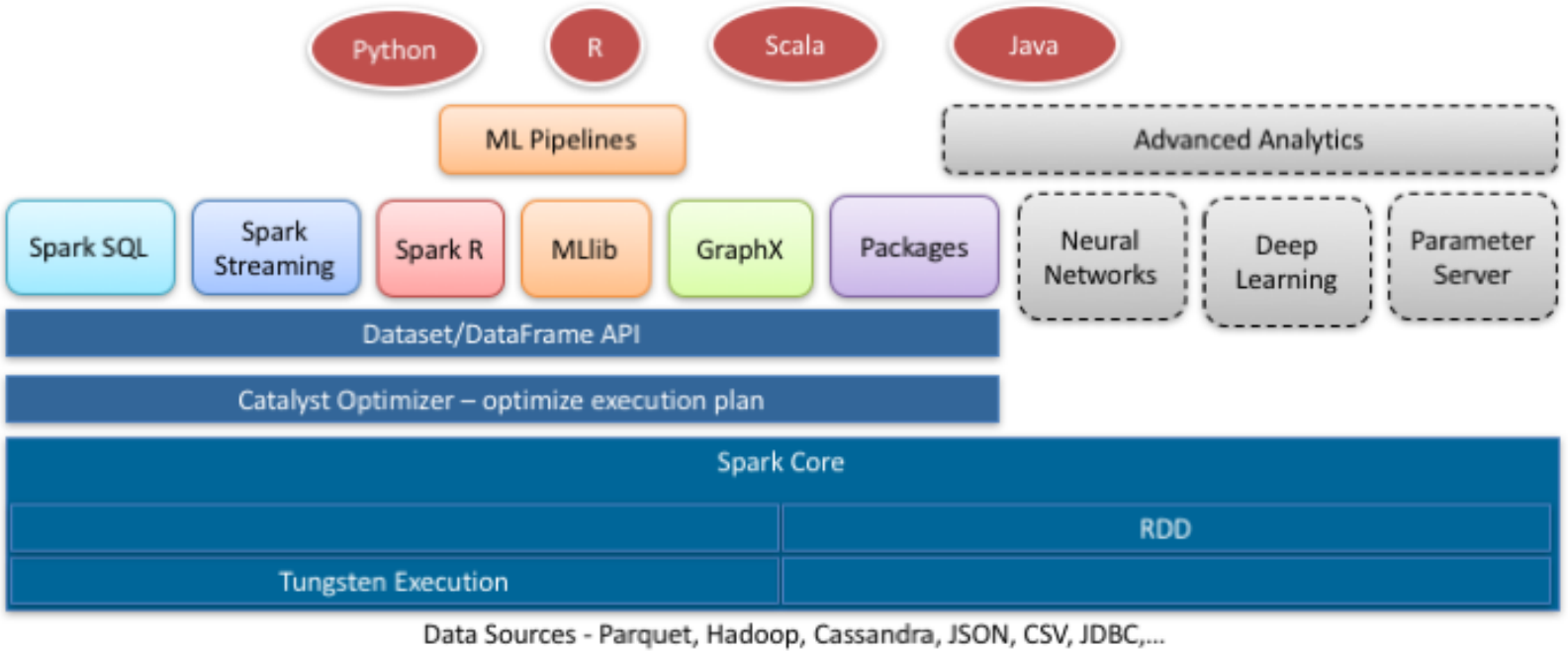


APACHE SPARK (3/4)



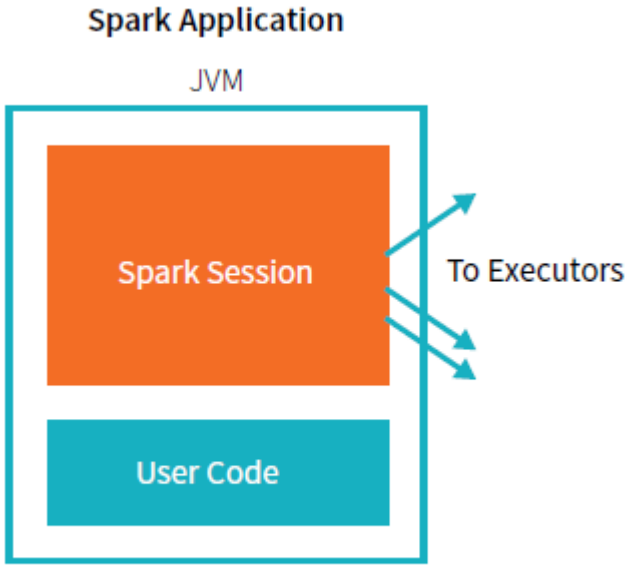
APACHE SPARK (4/4)

- Full stack



APLICACIONES SPARK (1/2)

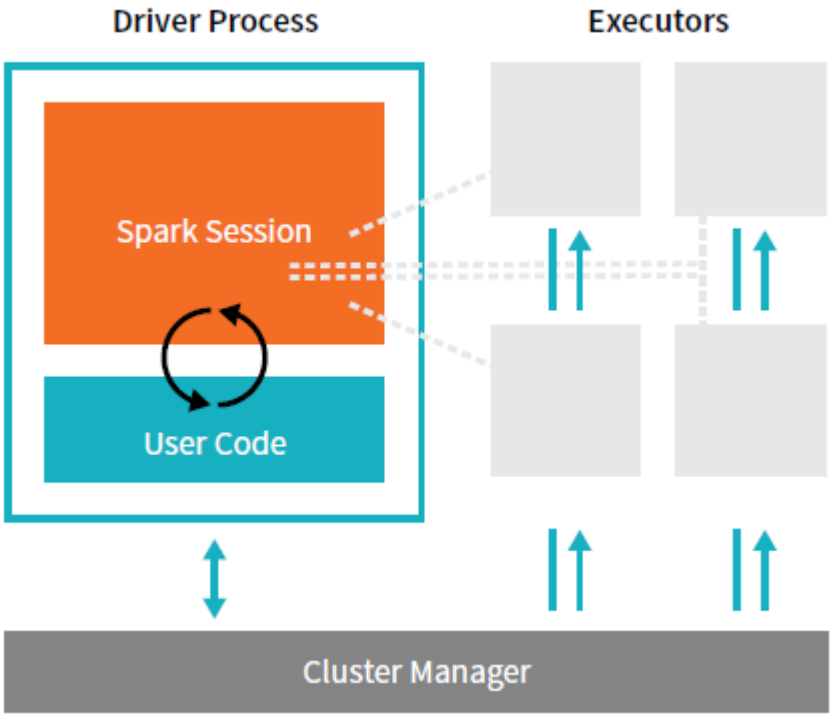
- Las aplicaciones Spark consisten en un proceso controlador (driver) y un conjunto de procesos ejecutores (executors).
  - El proceso controlador (driver) se ejecuta en un nodo del clúster (normalmente el máster) y es responsable de tres cosas:
    - Mantener la información sobre la aplicación Spark;
    - Responder al programa de un usuario;
    - Analizar, distribuir y programar el trabajo entre los ejecutores.



Figure

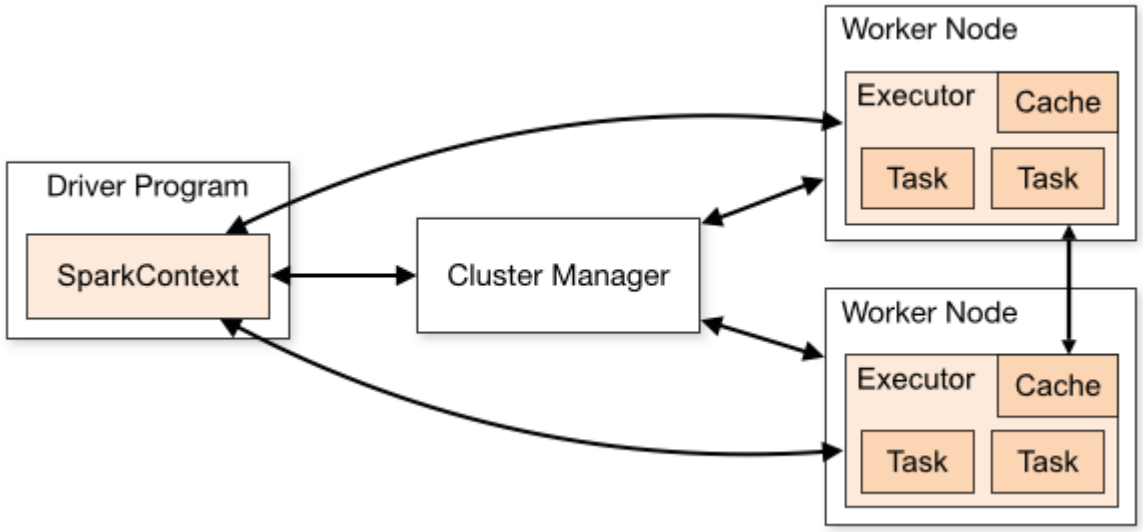
APLICACIONES SPARK (2/2)

- Los ejecutores son responsables de:
  - Ejecutar el código que se le ha asignado
  - Informar del estado del cálculo
- Spark soporta dos modos
  - local [N]
  - Cluster
    - Gestiona las máquinas/contenedores
    - Reserva de recursos para las aplicaciones Spark



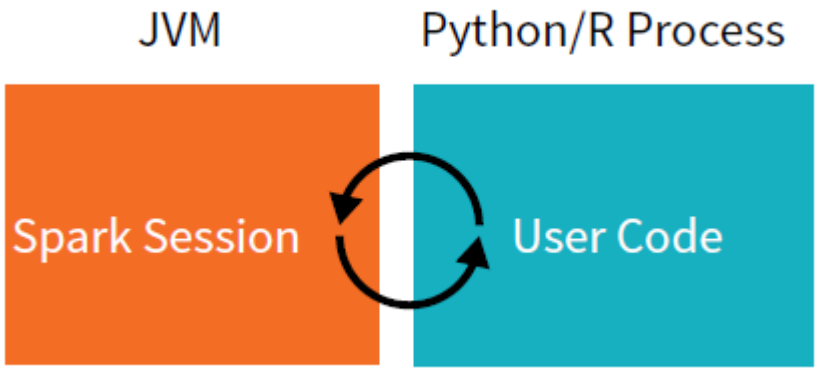
APACHE SPARK

- Cluster Managers
  - Standalone,
  - Apache Mesos
  - Hadoop YARN
  - EC (cloud)
  - Kubernetes



APIS DE LENGUAJE EN SPARK

- Scala/Java
  - Nativas, se ejecutan en la JVM
- SQL
- Python/R



- Objeto **SparkSession**
  - SparkSession será la representación de la aplicación (trabajo) para ejecutar el código Spark.
  - Python/R → el usuario nunca escribe instrucciones explícitas de JVM
    - El código se traduce de manera transparente (**rendimiento!!!**) que Spark puede ejecutar en las JVMs de los ejecutores.

INSTALACIÓN

- Pruebas/Desarrollo: Spark en una máquina simple -->> Pequeños Datasets
- Cluster dedicado (conexiones SSH)
  - Sin gestor de cluster externo → STANDALONE
  - Con gestor
    - Mesos: https://spark.apache.org/docs/latest/running-on-mesos.html
    - YARN: https://spark.apache.org/docs/latest/running-on-yarn.html
- Clusters en la nube
  - EC2: https://github.com/amplab/spark-ec2
  - Kubernetes: https://spark.apache.org/docs/latest/running-on-kubernetes.html
- Generación de Spark a partir del código fuente
  - Maven/Java

INSTALACIÓN (1/5)

- Seleccionar la versión, descomprimir…

Download Apache Spark™

1. Choose a Spark release: 2.2.1 (Dec 01 2017) ▾
2. Choose a package type: Pre-built for Apache Hadoop 2.7 and later ▾
3. Download Spark: spark-2.2.1-bin-hadoop2.7.tgz
4. Verify this release using the 2.2 signatures and checksums and project release KEYS.

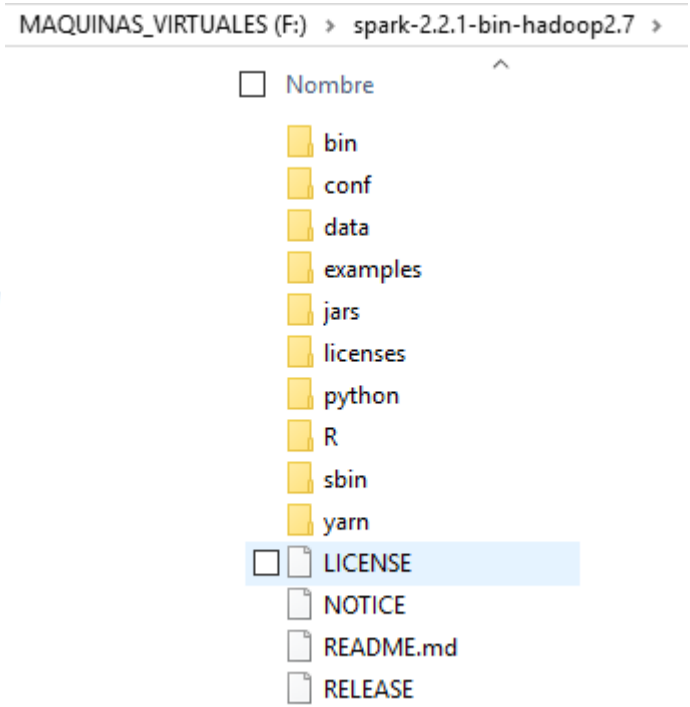
Note: Starting version 2.0, Spark is bu with Scala 2.11 by default. Scala 2.10 users should download build with Scala 2.10 support.

HTTP

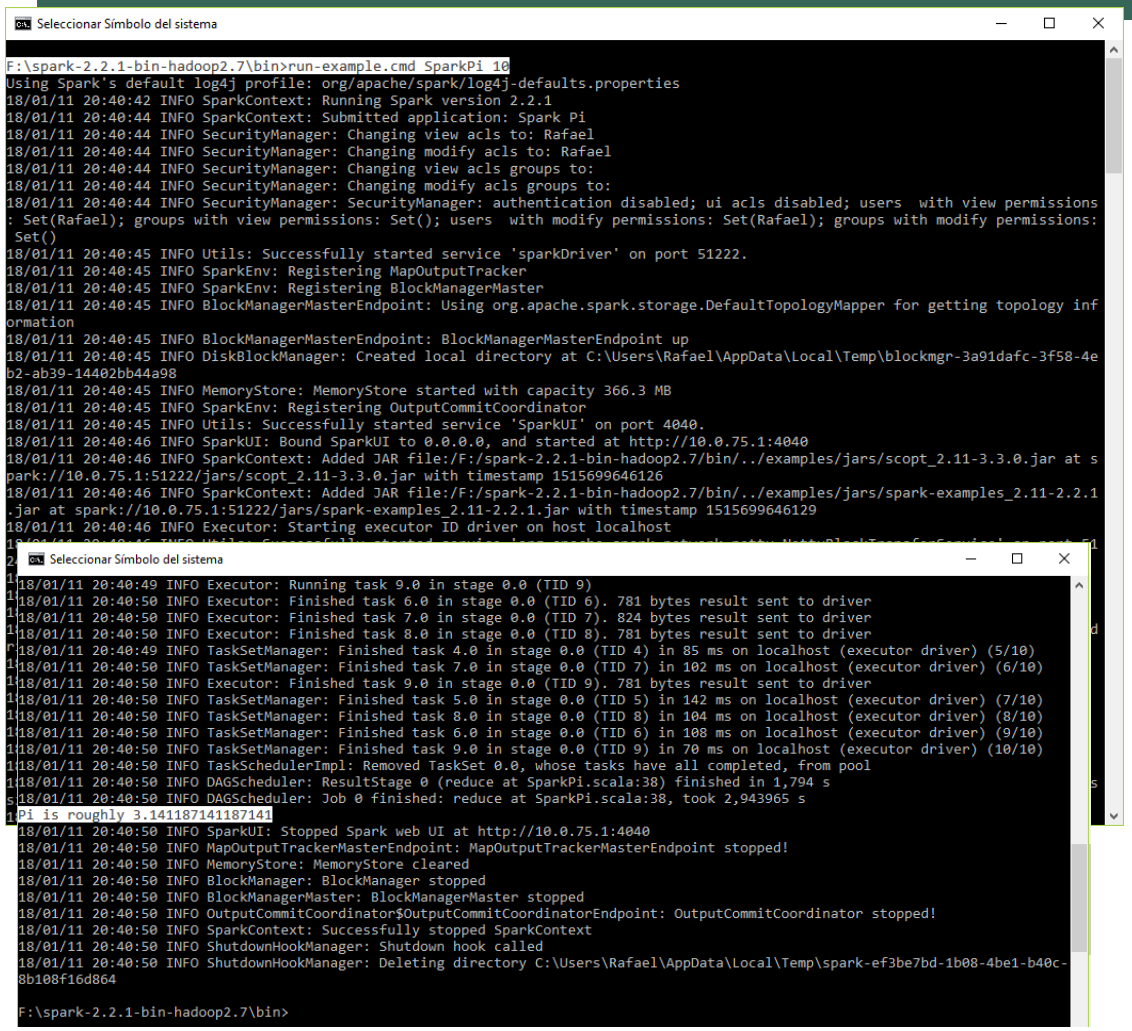
http://apache.rediris.es/spark/spark-2.2.1/spark-2.2.1-bin-hadoop2.7.tgz

http://apache.uvigo.es/spark/spark-2.2.1/spark-2.2.1-bin-hadoop2.7.tgz

http://ftp.cixug.es/apache/spark/spark-2.2.1/spark-2.2.1-bin-hadoop2.7.tgz



INSTALACIÓN (2/5)



Probar los ejemplos:

- Scala

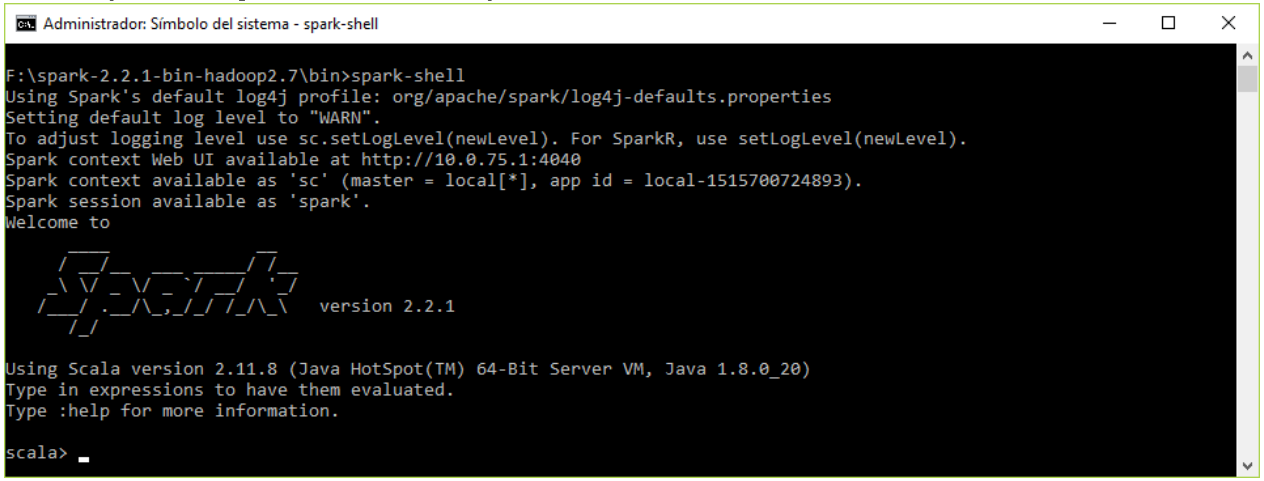
run-example SparkPi 10

- Python

spark-submit ..\examples\src\main\python\pi.py

INSTALACIÓN (3/5)

- Spark Shell (bin/spark-shell)




```
val textFile = sc.textFile("README.md")
val words = textFile.flatMap(line => line.split(" "))
val words_init_1 = words.map( word => (word,1))
val words_counted = words_init_1.reduceByKey( _+_ )
words_counted.collect()
words_counted.filter( words => words._2>10).take(10)
```

Apache Spark Examples

These examples give a quick overview of the Spark API. Spark is built on the concept of *distributed datasets*, which contain arbitrary Java or Python objects. You create a dataset from external data, then apply parallel operations to it. The building block of the Spark API is its RDD API. In the RDD API, there are two types of operations: *transformations*, which define a new dataset based on previous ones, and *actions*, which kick off a job to execute on a cluster. On top of Spark's RDD API, high level APIs are provided, e.g. DataFrame API and Machine Learning API. These high level APIs provide a concise way to conduct certain data operations. In this page, we will show examples using RDD API as well as examples using high level APIs.

RDD API Examples

Word Count

In this example, we use a few transformations to build a dataset of (String, Int) pairs called `counts` and then save it to a file.

Python | Scala | Java

```
text_file = sc.textFile("hdfs://...")
counts = text_file.flatMap(lambda line: line.split(" ")) \
             .map(lambda word: (word, 1)) \
             .reduceByKey(lambda a, b: a + b)
counts.saveAsTextFile("hdfs://...")
```

Pi Estimation

Spark can also be used for compute-intensive tasks. This code estimates π by "throwing darts" at a circle. We pick random points in the unit square ((0, 0) to (1,1)) and see how many fall in the unit circle. The fraction should be π / 4, so we use this to get our estimate.

Python | Scala | Java

```
def inside(p):
    x, y = random.random(), random.random()
    return x*x + y*y < 1

count = sc.parallelize(xrange(0, NUM_SAMPLES)) \
             .filter(inside).count()
print "Pi is roughly %f" % (4.0 * count / NUM_SAMPLES)
```

Latest News

Spark 2.4.3 released (May 08, 2019)
Spark 2.4.2 released (Apr 23, 2019)
Spark 2.4.1 released (Mar 31, 2019)
Spark 2.3.3 released (Feb 15, 2019)

Archive

APACHECON
LAS VEGAS: Sept. 9-12, 2019
BERLIN: Oct. 22-24, 2019

Download Spark

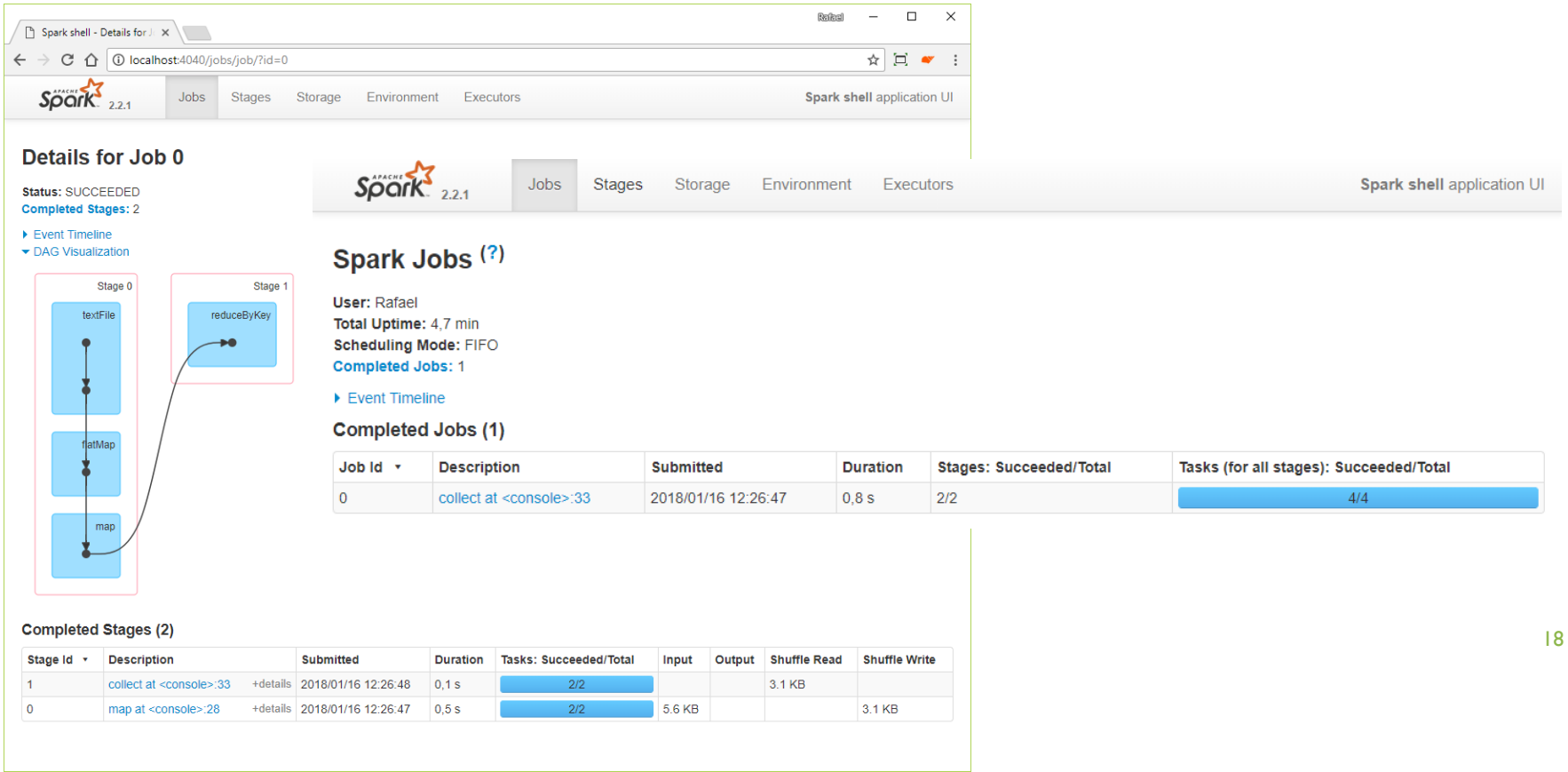
Built-in Libraries:

SQL and DataFrames
Spark Streaming
MLlib (machine learning)
GraphX (graph)

Third-Party Projects

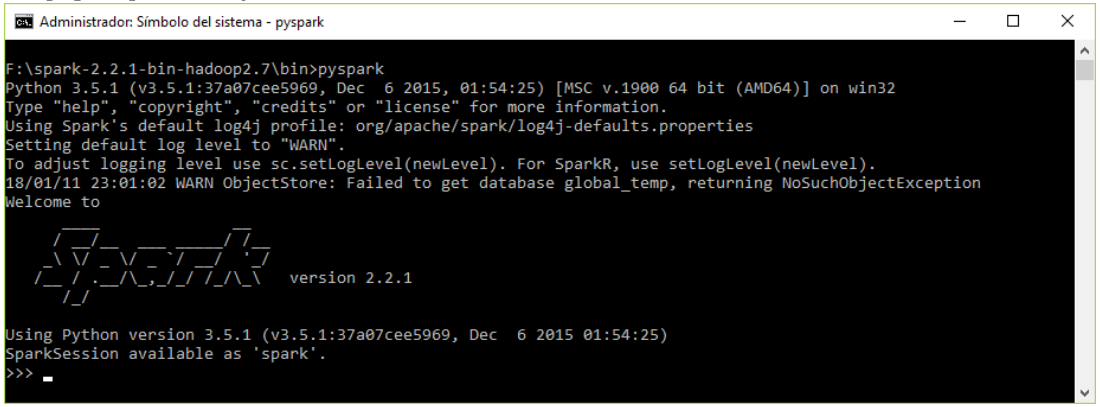
INSTALACIÓN (4/5)

- Monitor de trabajos → localhost:4040



INSTALACIÓN (5/5)

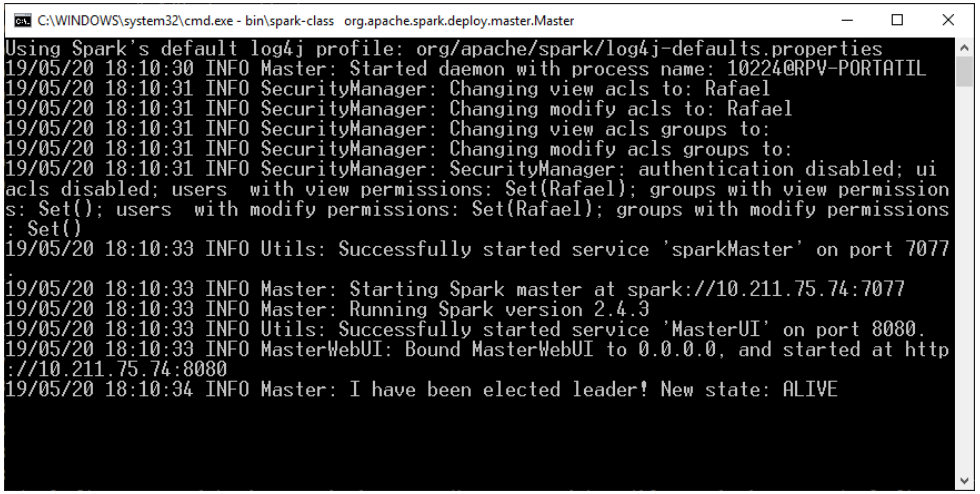
- PySpark (bin/pyspark)

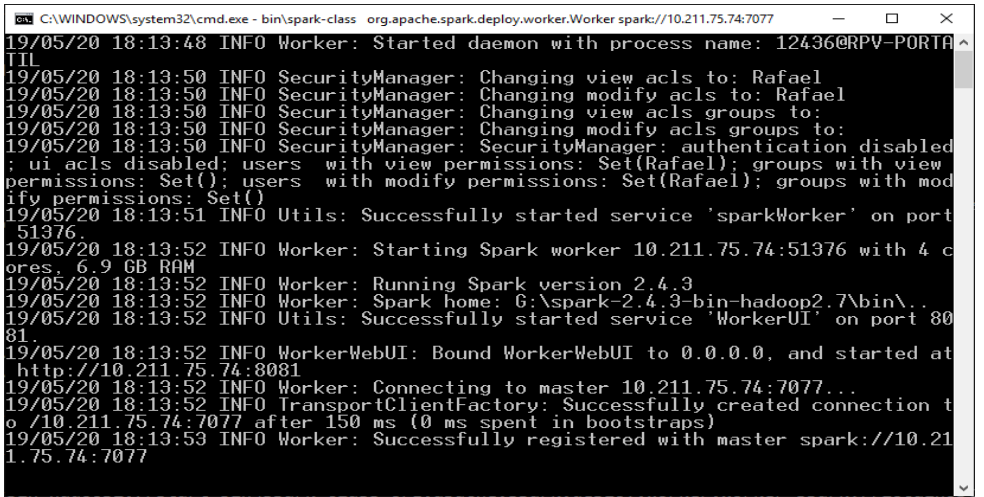



```
textFile = sc.textFile("README.md")
words = textFile.flatMap(lambda line: line.split(" "))
words_init_1 = words.map(lambda word: (word,1))
words_counted = words_init_1.reduceByKey(lambda a,b: a+b)
words_counted.collect()
words_counted.filter(lambda word: word[1]>10).take(10)
```

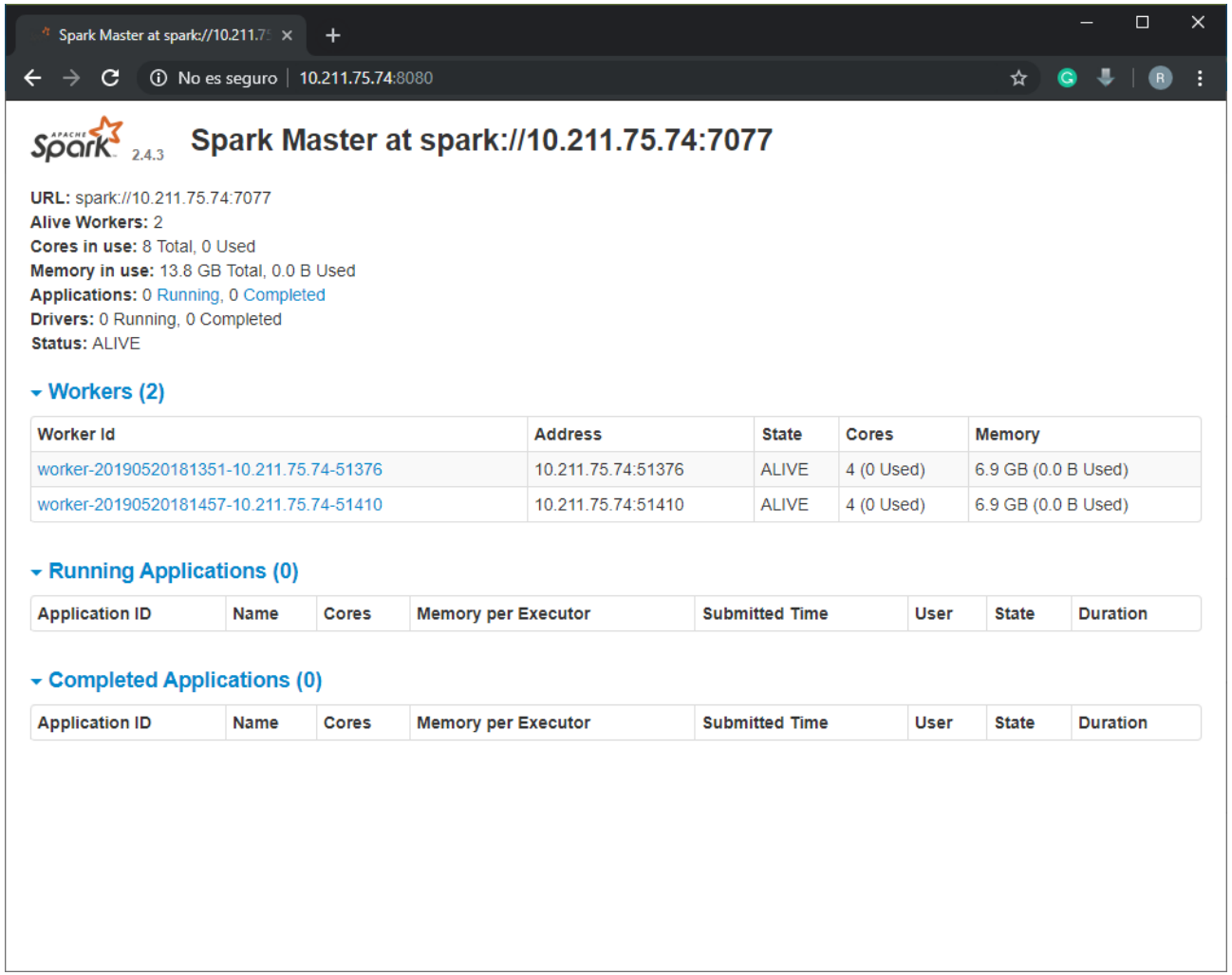
CLUSTER STANDALONE

- **sbin/** → start-master.sh y start-slave.sh (slaves)
- En Windows
- Master
  - `%SPARK_HOME%\bin\spark-class org.apache.spark.deploy.master.Master`
- Workers
  - `%SPARK_HOME%\bin\spark-class org.apache.spark.deploy.worker.Worker spark://ip:port`
  - El puerto por defecto es el 7077





CLUSTER STANDALONE: MASTER WEB UI



CLUSTER STANDALONE

- Ejecutar un ejemplo en el cluster
  - run-example --master spark://ip:port GroupByTest (Puerto por defecto 7077)
  - spark-submit --master spark://ip:port examples\ src\main\python\pi.py
- Conectar con el cluster en una sesión interactiva
  - spark-shell –master spark://ip:port
  - pyspark –master spark://ip:port